TECHNICAL BLUEPRINT · VERSION 2.0

Graph-Based RAG SOC AI Agent

A deterministic-first, cost-controlled Security Operations Center investigation assistant built on Neo4j, LangGraph, and a locally-hosted LLM.

DOCUMENT DATE
September 8, 2026

TEAM SIZE
6 Engineers

TIMELINE
8 Weeks

HARDWARE TARGET
CPU-Only (No Dedicated GPU)

Internal Engineering Document · Prepared as the project's technical blueprint · Annotated Edition with Delivery Insights

Table of Contents

1 System Architecture Overview

2 Core Design Principles

3 The Complete Tool Stack

4 Graph Schema Design

5 Cypher Query Library (The Pattern Engine)

6 The RAG Implementation

7 Pattern Recognition Engine

8 The ETL Pipeline

9 LLM Integration Without a GPU

10 Prompt Engineering & Analyst Briefing

11 The Adaptive False Positive Classifier

12 Cost-Effectiveness Strategy

13 Team Structure & Work Division

14 8-Week Implementation Roadmap

15 Risk Mitigation Strategies

16 Deployment & Demo Strategy

17 Appendix: Quick Start Commands

1. System Architecture Overview

The system is built as a modular pipeline that processes security alerts through deterministic pattern matching before invoking an LLM for complex cases. The architecture prioritizes speed, cost-effectiveness, and explainability.

1.1 The Complete Data Flow



1.2 How the Components Communicate

- **Wazuh** (SIEM) pushes alerts to the ETL pipeline via REST API.
- **Osquery** (EDR) provides process/network/file data via SQL queries.
- **OpenLDAP** (AD) provides user/group context via LDAP queries.
- **Neo4j** stores the graph and is queried via Cypher.
- **LangGraph** orchestrates the investigation: Retrieve → Match → Generate.
- **Ollama** runs the local LLM for summarization and recommendations.
- **FastAPI** serves the backend API.
- **React + Cytoscape.js** displays the dashboard and graph visualization.

2. Core Design Principles

These principles guide every engineering decision and are non-negotiable for the MVP.

2.1 Deterministic Tools (APIs)

The AI performs **read-only** retrieval and synthesis. Destructive/containment actions (e.g., isolating a host, revoking access) require **human approval** via a "One-Click Approve" interface. The system never executes remediation actions without explicit analyst confirmation.

2.2 Graph Representations

Attacks are represented as trees/graphs, not flat text. This structure makes context retrieval far more reliable for the LLM. The graph encodes process trees, user activity, network connections, and file operations as interconnected nodes.

2.3 Structured Outputs

The AI outputs structured JSON (using Pydantic schemas). This enables seamless triggering of SOAR workflows and consistent parsing by the UI.

2.4 Local LLM / Cost Control

To be production-viable, the system uses a local open-source LLM (Llama 3.1 8B via Ollama) instead of expensive 3rd-party APIs. Only 8-10% of alerts reach the LLM; 90% are resolved via deterministic pattern matching.

2.5 Human-in-the-Loop

The system augments, never replaces, the human analyst. The graph visualization and pattern matches provide full transparency, allowing the analyst to verify every recommendation before acting.

3. The Complete Tool Stack

#	Tool	Purpose
1	**Python**	Primary programming language for the entire project.
2	**LangChain**	Component library for building LLM workflows and retrievers.
3	**LangGraph**	Orchestrates multi-step, stateful agent investigation loops.
4	**Neo4j**	Graph database storing attack paths as nodes and edges.
5	**Cypher**	Query language for Neo4j; defines attack patterns.
6	**FastAPI**	Backend REST API framework serving the frontend.
7	**React**	Frontend framework for the SOC dashboard.
8	**Cytoscape.js**	Renders attack trees as interactive graphs in the UI.
9	**Wazuh**	Free, open-source SIEM providing security alerts.
10	**Osquery**	Free EDR; queries system processes and files via SQL.
11	**OpenLDAP**	Free Active Directory mock for user/group context.
12	**Ollama**	Runs open-source LLMs locally on CPU.
13	**Hugging Face**	Model hub and tokenizers for embeddings.
14	**Qdrant**	Vector database for storing past incident reports (RAG).
15	**XGBoost + ONNX**	ML classifier for false positive suppression.
16	**Celery**	Async task queue for long-running LLM investigations.
17	**Redis**	In-memory store; serves as Celery broker.
18	**Docker**	Containerizes all services for consistency.
19	**Git / GitHub**	Version control and code hosting.
20	**GitHub Actions**	CI/CD pipeline for automated testing and deployment.

4. Graph Schema Design

The graph schema defines how security data is structured in Neo4j. This schema is the foundation for all pattern matching queries.

4.1 Node Types (The "What")

Node Type	Properties	Example
`Alert`	`id`, `severity`, `name`, `timestamp`	`{id: "A-123", severity: "High", name: "Suspicious PowerShell"}`
`Process`	`pid`, `name`, `path`, `hash`, `timestamp`	`{pid: 4432, name: "powershell.exe", path: "C:\Windows\"}`
`Host`	`hostname`, `ip`, `os`	`{hostname: "WORKSTATION-01", ip: "192.168.1.100"}`
`User`	`username`, `domain`, `admin`	`{username: "jdoe", domain: "CONTOSO", admin: true}`
`Network`	`src_ip`, `dest_ip`, `dest_port`, `protocol`	`{dest_ip: "5.6.7.8", dest_port: 443, protocol: "TCP"}`
`File`	`path`, `hash`, `size`	`{path: "C:\Users\jdoe\passwd.txt", hash: "abc123"}`

4.2 Relationship Types (The "How")

Relationship	From	To	Meaning
`OWNS`	`User`	`Process`	User owns/started the process.
`SPAWNED`	`Process`	`Process`	Parent process spawned child process.
`RUNS_ON`	`Process`	`Host`	Process executed on this host.
`CONNECTED_TO`	`Process`	`Network`	Process made a network connection.
`READS`	`Process`	`File`	Process read a file.
`WRITES`	`Process`	`File`	Process wrote/created a file.
`TRIGGERED_BY`	`Alert`	`Process`	Alert was triggered by this process.
`LOGGED_IN`	`User`	`Host`	User logged into this host.
`SCANNED`	`Host`	`Host`	Host scanned another host.

4.3 Creating the Schema in Neo4j

```
// Create constraints and indexes for performance
CREATE CONSTRAINT FOR (a:Alert) REQUIRE a.id IS UNIQUE;
CREATE CONSTRAINT FOR (p:Process) REQUIRE p.pid IS UNIQUE;
CREATE CONSTRAINT FOR (h:Host) REQUIRE h.hostname IS UNIQUE;
CREATE CONSTRAINT FOR (u:User) REQUIRE u.username IS UNIQUE;
CREATE INDEX ON :Process(timestamp);
CREATE INDEX ON :Alert(timestamp);
```

5. Cypher Query Library (The Pattern Engine)

The pattern library consists of 20-25 hardcoded Cypher queries that map to common MITRE ATT&CK techniques. These queries are the primary detection mechanism.

5.1 The Pattern Library

Pattern ID	Pattern Name	MITRE Technique	Cypher Query
P-001	Phishing Payload	T1059.001	`MATCH (u:User)-[:OWNS]->(p:Process)-[:SPAWNED]->(c:Process) WHERE c.name CONTAINS "powershell" RETURN u, p, c`
P-002	Lateral Movement	T1021	`MATCH (u:User)-[:LOGGED_IN]->(h1:Host)-[:CONNECTED_TO]->(h2:Host) RETURN u, h1, h2`
P-003	Credential Dumping	T1003.002	`MATCH (p:Process)-[:READS]->(f:File) WHERE f.path CONTAINS "SAM" OR f.path CONTAINS "NTDS" RETURN p, f`
P-004	C2 Beaconing	T1071.001	`MATCH (p:Process)-[:CONNECTED_TO]->(n:Network) WHERE n.dest_port IN [443, 80, 53] AND NOT n.dest_ip STARTS WITH "192.168." RETURN p, n`
P-005	Privilege Escalation	T1068	`MATCH (u:User {admin: false})-[:OWNS]->(p:Process)-[:SPAWNED]->(c:Process) WHERE c.name CONTAINS "admin" RETURN u, p, c`
P-006	Ransomware Behavior	T1486	`MATCH (p:Process)-[:WRITES]->(f:File) WHERE f.extension IN ['.encrypted', '.locked'] RETURN p, f`
P-007	Data Exfiltration	T1048	`MATCH (p:Process)-[:READS]->(f:File)-[:CONNECTED_TO]->(n:Network) RETURN p, f, n`
P-008	Persistence Mechanism	T1547	`MATCH (p:Process)-[:WRITES]->(f:File) WHERE f.path CONTAINS "Startup" OR f.path CONTAINS "Run" RETURN p, f`
P-009	Network Reconnaissance	T1046	`MATCH (h:Host)-[:SCANNED]->(h2:Host) RETURN h, h2`
P-010	Golden Ticket Attack	T1558.001	`MATCH (u:User)-[:OWNS]->(p:Process) WHERE p.name CONTAINS "mimikatz" AND p.args CONTAINS "krbtgt" RETURN u, p`
P-011	Kerberoasting	T1558.003	`MATCH (p:Process) WHERE p.name CONTAINS "GetUserSPNs" OR p.args CONTAINS "kerberoast" RETURN p`
P-012	Remote Execution	T1569.002	`MATCH (p:Process)-[:SPAWNED]->(c:Process) WHERE c.name CONTAINS "wmic" OR c.name CONTAINS "psexec" RETURN p, c`
P-013		T1112	

Pattern ID	Pattern Name	MITRE Technique	Cypher Query
	Registry Modification		`MATCH (p:Process)-[:WRITES]->(f:File) WHERE f.path CONTAINS "reg.exe" OR f.path CONTAINS ".reg" RETURN p, f`
P-014	Scheduled Task Creation	T1053.005	`MATCH (p:Process)-[:WRITES]->(f:File) WHERE f.path CONTAINS "schtasks" RETURN p, f`
P-015	Service Creation	T1543.003	`MATCH (p:Process)-[:SPAWNED]->(c:Process) WHERE c.name CONTAINS "sc.exe" OR c.args CONTAINS "create" RETURN p, c`
P-016	PowerShell Download Cradle	T1059.001	`MATCH (p:Process) WHERE p.name CONTAINS "powershell" AND (p.args CONTAINS "Invoke-WebRequest" OR p.args CONTAINS "DownloadString") RETURN p`
P-017	Port Scanning	T1046	`MATCH (h:Host)-[:CONNECTED_TO]->(n:Network) WHERE n.dest_port IN [22, 23, 80, 443, 445, 3389] AND n.dest_ip <> "127.0.0.1" RETURN h, n`
P-018	Password Spraying	T1110.003	`MATCH (u:User) WHERE u.failed_logins > 10 AND u.last_failed > timestamp() - 3600 RETURN u`
P-019	Supply Chain Compromise	T1195	`MATCH (p:Process)-[:READS]->(f:File) WHERE f.path CONTAINS "compromised-update" RETURN p, f`
P-020	LLMNR/NBT-NS Spoofing	T1557.001	`MATCH (n:Network) WHERE n.protocol IN ["LLMNR", "NBT-NS"] AND n.src_ip <> n.dest_ip RETURN n`

5.2 Pattern Matching Execution

```
def match_patterns(alert_id):
    matched = []
    for pattern in PATTERN_LIBRARY:
        result = neo4j.run(pattern['cypher'] + f" WHERE a.id = '{alert_id}'")
        if result:
            matched.append(pattern)
    return matched
```

6. The RAG Implementation

The system uses Retrieval-Augmented Generation at two layers: graph retrieval and vector retrieval.

6.1 Graph Retrieval (Live Data)

The primary retrieval source is Neo4j. When an alert arrives, the system retrieves the subgraph surrounding the alert:

```
MATCH (a:Alert {id: 'A-123'})-[*1..3]-(connected)
RETURN a, connected
```

This retrieves all nodes within 3 relationships of the alert, providing complete process trees, user context, and network connections.

6.2 Vector Retrieval (Historical Knowledge)

Past incident reports and public security knowledge are stored in Qdrant:

```
from qdrant_client import QdrantClient

client = QdrantClient(host="localhost", port=6333)

# Retrieve similar past incidents
results = client.search(
    collection_name="incident_reports",
    query_vector=embedding_of_alert,
    limit=3
)

# Results contain past RCAs, MITRE mappings, and containment steps
```

6.3 The Complete RAG Loop

```
Alert → Graph Retrieval (Neo4j) → Pattern Matching → Vector Retrieval (Qdrant) → LLM Generation → Analyst Briefing
```

7. Pattern Recognition Engine

The pattern recognition engine is the core of the system. It doesn't rely on the LLM for detection—it uses deterministic Cypher queries.

7.1 How Pattern Recognition Works

1. **Alert triggers** the investigation.
2. **Engine runs all 20 Cypher queries** against the Neo4j graph.
3. **Matches are collected.**
    - 3+ matches → HIGH confidence
    - 1-2 matches → MEDIUM confidence
    - 0 matches → LOW confidence
4. **LLM is only invoked** for MEDIUM and HIGH confidence alerts (approx 8-10% of total).

7.2 Pattern Scoring Logic

```
def evaluate_patterns(matched_patterns):
    confidence = "LOW"
    if len(matched_patterns) >= 3:
        confidence = "HIGH"
    elif len(matched_patterns) >= 1:
        confidence = "MEDIUM"

    # Check for kill chain (multiple patterns on same host)
    kill_chain = False
    if "P-001" in matched and "P-003" in matched and "P-007" in matched:
        kill_chain = True
        confidence = "CRITICAL"

    return confidence, kill_chain
```

7.3 The Kill Chain Detector

The system can detect multi-stage attacks by correlating patterns across time:

```
// Find alerts on the same host within 30 minutes
MATCH (a1:Alert)-[:TRIGGERED_BY]->(p1:Process)-[:RUNS_ON]->(h:Host)
MATCH (a2:Alert)-[:TRIGGERED_BY]->(p2:Process)-[:RUNS_ON]->(h:Host)
WHERE a1.timestamp < a2.timestamp AND a2.timestamp < a1.timestamp + 1800
RETURN h, collect(a1), collect(a2)
```

8. The ETL Pipeline

The ETL pipeline pulls data from Wazuh, Osquery, and OpenLDAP, transforms it, and loads it into Neo4j.

8.1 The ETL Flow

```
# Step 1: Query Wazuh for new alerts
alerts = wazuh_api.get_alerts(time_range="5m")

for alert in alerts:
    # Step 2: Extract the PID and Host
    pid = alert['process']['pid']
    host = alert['host']['hostname']

    # Step 3: Query Osquery for the process tree
    process_tree = osquery_api.query_processes(pid=pid, depth=3)

    # Step 4: Query OpenLDAP for the user
    user = ldap_api.get_user_by_sid(alert['user']['sid'])

    # Step 5: Build Cypher queries dynamically
    cypher_queries = [
        f"CREATE (a:Alert {{id: '{alert['id']}', severity: '{alert['severity']}'}})",
        f"CREATE (p:Process {{pid: {pid}, name: '{process_tree['name']}'}})",
        f"CREATE (u:User {{username: '{user['username']}', admin: {user['admin']}}})",
        f"CREATE (h:Host {{hostname: '{host}'}})",
        f"CREATE (a)-[:TRIGGERED_BY]->(p)",
        f"CREATE (u)-[:OWNS]->(p)",
        f"CREATE (p)-[:RUNS_ON]->(h)",
    ]

    # Step 6: Execute all Cypher queries on Neo4j
    for query in cypher_queries:
        neo4j_client.run(query)
```

8.2 Handling Large Process Trees

To avoid graph bloat, the system trims the process tree to +/- 2 parent levels and +/- 2 child levels:

```
def get_process_tree(pid, depth=2):
    query = f"""
        SELECT * FROM processes
        WHERE pid = {pid}
        OR parent = {pid}
        OR pid IN (SELECT parent FROM processes WHERE pid = {pid})
    """
    # Additional recursion for depth
```

9. LLM Integration Without a GPU

Running Llama 3.1 on CPU-only hardware requires specific strategies.

9.1 CPU Inference Setup

Install Ollama:

```
curl -fsSL https://ollama.com/install.sh | sh
```

Pull a Quantized Model:

```
# Q4_K_M is the best balance for CPU (4.9 GB RAM)
ollama pull llama3.1:8b-q4_0
```

Force CPU-Only Mode:

```
# Option 1: Environment variable
export OLLAMA_NUM_GPU=0

# Option 2: Create a Modelfile
FROM llama3.1:8b-q4_0
PARAMETER num_gpu 0

# Create the model
ollama create llama3.1-cpu -f Modelfile
```

9.2 Performance Expectations

Hardware	Tokens/Second	Time for 500-word Briefing
Modern CPU (e.g., Intel i7)	3-8 tokens/sec	80-200 seconds
High-end CPU (e.g., AMD Threadripper)	10-15 tokens/sec	40-60 seconds
Dedicated GPU (RTX 4090)	30-50 tokens/sec	10-15 seconds

9.3 Optimizing for CPU Inference

Use the `--num-gpu` Flag:

```
llm = Ollama(
    model="llama3.1:8b-q4_0",
    num_gpu=0,       # Force CPU
```

```
    num_thread=8,    # Use all CPU cores
)
```

Reduce Context Window:
Since the system provides condensed graph data, use a smaller context window (2048 tokens instead of 8192) to speed up inference.

Use the `num_keep` Parameter:
Keep the system prompt in memory to avoid reprocessing it.

9.4 Demonstration Strategy

Since CPU inference is slower, structure the demo to hide the latency:

1. **Immediate:** Display the graph visualization (instant).
2. **Immediate:** Show the matched patterns (instant).
3. **Background:** Generate the LLM summary (30-60 seconds).
4. **When ready:** Display the briefing + recommendations.

During the demo: While the LLM is generating, verbally walk through the graph and patterns. By the time you finish, the briefing is ready.

10. Prompt Engineering & Analyst Briefing

10.1 The Input (JSON from Graph + Patterns)

```
{
  "alert": { "id": "A-123", "severity": "High" },
  "graph": {
    "nodes": [
      { "type": "User", "name": "Bob", "admin": true },
      { "type": "Process", "name": "powershell.exe", "pid": 4432 },
      { "type": "Network", "dest_ip": "5.6.7.8", "port": 443 }
    ],
    "relationships": [
      { "from": "Bob", "to": "powershell.exe", "type": "OWNS" },
      { "from": "powershell.exe", "to": "5.6.7.8", "type": "CONNECTED_TO" }
    ]
  },
  "matched_patterns": [
    { "pattern": "P-001", "name": "Phishing Payload" },
    { "pattern": "P-004", "name": "C2 Beaconing" }
  ]
}
```

10.2 The Prompt Template

```
prompt = f"""
You are a Senior SOC Analyst AI Assistant.
Your goal is to help the human analyst make a decision in under 10 seconds.

## ALERT DETAILS
Alert ID: {alert_id}
Severity: {severity}
Alert Type: {alert_type}

## ATTACK GRAPH (Live Data)
User: {user_name} (Admin: {is_admin})
Process: {process_name} (PID: {pid})
Network: Connected to {dest_ip} on port {dest_port}

## MATCHED PATTERNS
{matched_patterns_list}

## TASK
Provide an executive summary, threat level, MITRE techniques, recommendations, and one-click
actions in JSON format.

## OUTPUT FORMAT (Strict JSON)
{{
  "summary": "One-sentence explanation for the analyst",
  "threat_level": "CRITICAL|HIGH|MEDIUM|LOW",
```

```
  "confidence": 92,
  "mitre_techniques": ["T1059.001 (PowerShell)", "T1071.001 (C2 Communication)"],
  "recommendations": [
    "Isolate host WORKSTATION-01 immediately.",
    "Revoke Bob's Active Directory tokens."
  ],
  "one_click_actions": [
    {{"action": "isolate_host", "params": {{"hostname": "WORKSTATION-01"}}}},
    {{"action": "revoke_tokens", "params": {{"username": "Bob"}}}}
  ]
}}
"""
```

10.3 The Output (Analyst Briefing)

```
{
  "summary": "User Bob ran a suspicious PowerShell that connected to a known malicious C2
server at 5.6.7.8.",
  "threat_level": "CRITICAL",
  "confidence": 92,
  "mitre_techniques": ["T1059.001 (PowerShell)", "T1071.001 (C2 Communication)"],
  "recommendations": [
    "Isolate host WORKSTATION-01 immediately.",
    "Revoke Bob's Active Directory tokens.",
    "Reset Bob's password."
  ],
  "one_click_actions": [
    { "action": "isolate_host", "params": { "hostname": "WORKSTATION-01" } },
    { "action": "revoke_tokens", "params": { "username": "Bob" } }
  ]
}
```

10.4 The UI Display

The React frontend renders the briefing with buttons:

```
┌──────────────────────────────────────────────────────────┐
│  CRITICAL ALERT: A-123                                   │
│  User Bob (Admin) ran PowerShell → C2 5.6.7.8            │
│                                                          │
│  [ Attack Graph]  [ MITRE T1059.001, T1071.001]          │
│                                                          │
│  Recommendations:                                        │
│  [ Isolate Host]  [ Revoke Tokens]  [ Escalate]          │
└──────────────────────────────────────────────────────────┘
```

11. The Adaptive False Positive Classifier

11.1 Purpose

The classifier uses machine learning to identify and auto-close benign alerts, reducing analyst workload.

11.2 Training Data

Historical alert data with labels from past analyst decisions:

- `alert_type`: The type of alert (e.g., "Suspicious PowerShell")
- `time_of_day`: Hour of the day (0-23)
- `day_of_week`: Day of the week (1-7)
- `source_ip`: Source IP address
- `destination_ip`: Destination IP address
- `user_admin`: Boolean indicating if the user is an admin
- `previous_alerts`: Number of similar alerts from this host in the last 24 hours
- `label`: 0 (benign/false positive) or 1 (true positive)

11.3 Model Training

```
import xgboost as xgb
from sklearn.model_selection import train_test_split

# Load historical data
X_train, X_test, y_train, y_test = train_test_split(
    features, labels, test_size=0.2, random_state=42
)

# Train XGBoost
model = xgb.XGBClassifier(
    n_estimators=100,
    max_depth=5,
    learning_rate=0.1,
    objective='binary:logistic'
)
model.fit(X_train, y_train)

# Save model
model.save_model('fp_classifier.onnx')
```

11.4 Inference Pipeline

```
# Load the ONNX model
import onnxruntime as ort
session = ort.InferenceSession('fp_classifier.onnx')

# Predict for new alert
features = extract_features(alert)
prediction = session.run(None, {'input': features})
false_positive_prob = prediction[0][0]

if false_positive_prob > 0.95:
    # Auto-close the alert
    auto_close(alert_id)
```

12. Cost-Effectiveness Strategy

The system minimizes costs by using deterministic pattern matching for most alerts and local LLM inference for the rest.

12.1 Cost Breakdown

Layer	% of Alerts	LLM Used?	Cost per Alert
Pattern Matching	90%	No	$0.00
One-Click Approve (LLM)	8%	Yes (Local)	~$0.0001 (electricity)
Escalate to Human	2%	No	$0.00

12.2 Electricity Cost (CPU Inference)

A CPU running at 65W for 1 hour uses 0.065 kWh. At $0.15/kWh, this costs about $0.01 per hour. If the CPU runs the LLM for 2 hours per day, the monthly cost is ~$0.60.

Result: The system handles 3 million alerts/month for <$1 in electricity costs.

13. Team Structure & Work Division

13.1 The 6 Roles

Role	Title	Core Responsibility	Week 1-2 Deliverable
1	**Orchestrator (Team Lead)**	LangGraph agent logic, prompt engineering, LLM integration.	The agent receives an alert, calls the graph, and outputs a JSON summary.
2	**Graph Engineer**	Neo4j schema design, Cypher query library, graph algorithms.	20 hardcoded Cypher queries and the finalized Graph Schema.
3	**Data/ETL Engineer**	Wazuh, Osquery, OpenLDAP setup; Python pipelines to pull data into Neo4j.	Live data flowing from mock SIEM/EDR into Neo4j every 5 minutes.
4	**ML/Classifier Engineer**	XGBoost false-positive classifier + Vector DB (Qdrant) for RAG retrieval.	A working ML model that auto-closes 50% of benign alerts.
5	**Backend API Engineer**	FastAPI endpoints, Celery/Redis async tasks, and SOAR webhook triggers.	A `/investigate` endpoint that triggers the LangGraph agent and returns JSON.
6	**Frontend/UI Engineer**	React dashboard, Cytoscape.js graph visualization, and "One-Click Approve" UI.	A clickable UI that displays the attack graph and the AI's recommendation.

13.2 Daily Status: Written Reports + Twice-Weekly Video Syncs

In place of a daily 15-minute standup call, each of the 6 engineers submits a **1–2 page written daily status report** covering the same core information a standup would surface, but with more depth and a permanent record the whole team (and any outside reviewer) can search later:

- **What I completed yesterday** — specific, verifiable progress (commits, queries written, endpoints shipped, tests passed), not a vague activity summary.
- **What I'm working on today** — the concrete task(s) for the day, tied back to the current week's milestone from Section 14.
- **Any blockers preventing progress** — named explicitly, with who/what is needed to unblock, so blockers don't sit silently until the next sync.

These writeups are posted daily (e.g., in the `#soc-ai-dev` channel referenced in Section 15, or a shared doc/repo folder) so any team member can catch up asynchronously at any time, without needing everyone online at the same moment.

Live video syncs are held twice a week — every Monday and Friday — to present the overall status of the project as a group:

- **Monday sync:** sets direction for the week — review the weekend/prior week's writeups, confirm the week's goals from the roadmap in Section 14, and surface any cross-role dependencies before work begins.

- **Friday sync:** closes the loop — each role demos or reports what shipped that week, flags anything slipping into the following week, and the team collectively decides on any re-prioritization.

14. 8-Week Implementation Roadmap

14.1 Phase 1: Foundation (Weeks 1-2)

Week	Focus	Team Tasks
Week 1	**Graph Schema & Data Ingestion**	*Role 2*: Whiteboard graph schema, write 20 Cypher queries. *Role 3*: Spin up Wazuh, Osquery, OpenLDAP in Docker. *Role 6*: Setup React + Cytoscape.js skeleton.
Week 2	**ETL Pipeline & API**	*Role 3*: Write Python ETL to pull data into Neo4j. *Role 5*: Create FastAPI `/investigate` endpoint. *Role 4*: Load historical data, train XGBoost baseline.

14.2 Phase 2: Integration & Brain (Weeks 3-4)

Week	Focus	Team Tasks
Week 3	**LangGraph Agent**	*Role 1*: Build the ReAct agent loop (Retrieve → Match → Generate). *Role 4*: Build Vector DB (Qdrant) and ingest 50 public incident reports.
Week 4	**LLM Integration & UI**	*Role 1*: Integrate Ollama + Llama 3.1 8B (CPU mode). *Role 6*: Wire UI to `/investigate` API; render graph + briefing. *Role 5*: Deploy to staging.

14.3 Phase 3: Polish & Hardening (Weeks 5-6)

Week	Focus	Team Tasks
Week 5	**Prompt Engineering**	*Role 1*: Fine-tune prompts (few-shot examples, LangSmith tracing). *Role 2*: Write advanced correlation queries (time-based).
Week 6	**Incident Scribe**	*Role 1*: Build the Scribe (auto-generates RCA reports). *Role 6*: Add "Investigation Timeline" component. *Role 3*: Add File node support.

14.4 Phase 4: Demo & Cutover (Weeks 7-8)

Week	Focus	Team Tasks
Week 7	**Performance Tuning**	*Role 2*: Optimize Cypher queries (add indexes). *Role 4*: Final model evaluation (precision/recall). *Role 5*: Add WebSocket support for real-time alerts.

Week	Focus	Team Tasks
Week 8	**UAT & Demo**	*All roles*: End-to-end testing on 20 historical alerts. *Role 6*: Record 5-minute demo video. *Role 5*: Deploy to public URL (Render.com / Fly.io).

15. Risk Mitigation Strategies

Risk	Impact	Mitigation
API Access Delays	Can't pull live data without APIs.	Use Docker containers and mock data (Splunk BOTS dataset) until APIs are ready.
Graph Bloat	Too many nodes in a single query.	Trim graph to +/- 2 parent levels and +/- 2 child levels from the malicious PID.
Prompt Injection	Attacker names a process `Ignore all previous instructions`.	Sanitize log strings (strip special chars) before putting them in the prompt. Never let LLM output execute code directly.
LLM Hallucination	LLM invents MITRE IDs or incorrect containment.	Use hardcoded MITRE lookup table; display the graph visually so the analyst can verify.
Team Misalignment	Engineers work in silos.	Daily written status reports (1–2 pages per engineer), with live video syncs every Monday and Friday. Use `#soc-ai-dev` channel for real-time updates.
CPU Inference Slowness	LLM takes 60-200 seconds to respond.	Use async tasks (Celery) so UI doesn't block. Generate briefing in the background.

16. Deployment & Demo Strategy

16.1 Deployment to a Public URL

Since you're running locally, you'll need to deploy to a cloud provider for demos:

Option A: Render.com (Free Tier)

- Deploy FastAPI backend as a Web Service.
- Neo4j, Redis, and Qdrant can be deployed as Docker containers.

Option B: Fly.io (Free Tier)

- Good for Docker-based deployments.
- Supports persistent volumes for Neo4j.

Option C: Self-Hosted with ngrok

- Run everything locally.
- Use `ngrok` to expose your local FastAPI server to the internet.

16.2 Demo Day Checklist

1. All services running (Wazuh, Osquery, Neo4j, Ollama, FastAPI, React).
2. 5-10 pre-loaded alerts ready for investigation.
3. Attack graphs rendering correctly in Cytoscape.js.
4. LLM generating briefings (even if slow).
5. One-click approve buttons working.
6. 5-minute screen recording ready as backup.

17. Appendix: Quick Start Commands

17.1 Environment Setup

```
# Python virtual environment
python -m venv venv
source venv/bin/activate   # Linux/Mac
venv\Scripts\activate      # Windows

# Install core libraries
pip install langchain langgraph fastapi uvicorn transformers
pip install accelerate tokenizers celery redis
```

17.2 Container Setup

```
# Neo4j
docker run -d -p 7474:7474 -p 7687:7687 --name neo4j neo4j:latest

# Redis
docker run -d -p 6379:6379 --name redis redis:alpine

# Wazuh (full stack)
curl -s https://packages.wazuh.com/4.x/wazuh-install.sh | bash

# OpenLDAP
docker run -d -p 389:389 -p 636:636 --name ldap osixia/openldap:latest

# Qdrant
docker run -d -p 6333:6333 --name qdrant qdrant/qdrant
```

17.3 Ollama Setup (CPU Mode)

```
# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Pull quantized model
ollama pull llama3.1:8b-q4_0

# Create CPU-only model
echo "FROM llama3.1:8b-q4_0\nPARAMETER num_gpu 0" > Modelfile
ollama create llama3.1-cpu -f Modelfile

# Test the model
ollama run llama3.1-cpu "Hello, world!"
```

17.4 Frontend Setup

```
# Create React app
npx create-react-app soc-dashboard
cd soc-dashboard

# Install Cytoscape
npm install cytoscape cytoscape-dagre

# Start the dev server
npm start
```

17.5 Quick Test Script

```
# test_etl.py - Quick test of ETL pipeline
import requests

# Test Wazuh connection
wazuh_status = requests.get("http://localhost:55000/", verify=False)
print(f"Wazuh: {wazuh_status.status_code}")

# Test Osquery connection
osquery_status = requests.post("http://localhost:8080/api/v1/query",
    json={"query": "SELECT 1"})
print(f"Osquery: {osquery_status.status_code}")

# Test Neo4j connection
neo4j_status = requests.get("http://localhost:7474")
print(f"Neo4j: {neo4j_status.status_code}")

# Test Ollama connection
ollama_status = requests.post("http://localhost:11434/api/generate",
    json={"model": "llama3.1-cpu", "prompt": "Hello"})
print(f"Ollama: {ollama_status.status_code}")
```

— End of Document —

Graph-Based RAG SOC AI Agent · Technical Implementation Guide v2.0 · Annotated Edition